Seguridad y Alta Disponibilidad (2º ASIR)

Ejercicios · UD1 Adopción de pautas de seguridad informática · Fundamentos de criptografía

Fecha	Martes 06/10/2026 · 15:00–16:50
Contenido trabajado	Diapositivas 43–47 · PDF pp. 38–41
Temas	Concepto y componentes de la criptografía · cifrado simétrico · cifrado por bloques y modos de operación · algoritmos simétricos
Nombre y apellidos	

Ejercicio 1 · Componentes de un sistema criptográfico

Dificultad: Básica · Tiempo orientativo: 20 min

1. Define estos términos con tus palabras: **texto en claro**, **criptograma**, **algoritmo de cifrado**, **clave**.

Texto en claro: Es el mensaje original que se puede leer y entender antes de cifrarlo.

Criptograma: Es el mensaje que se obtiene después de aplicar un algoritmo de cifrado. No se puede entender directamente sin descifrarlo.

Algoritmo de cifrado: Es el conjunto de reglas matemáticas que se utiliza para transformar un mensaje en claro en un mensaje cifrado.

Clave: Es el dato que utiliza el algoritmo para cifrar o descifrar un mensaje.

2. Usa el **cifrado César con desplazamiento 3** (alfabeto inglés de 26 letras, sin Ñ):

a) Cifra la palabra: `SEGURIDAD`

```
S → V, E → H, G → J, U → X, R → U, I → L, D → G, A → D, D → G.

Resultado: VHJXULGDG
```

b) Descifra el criptograma: `FLIUDGR`

```
Para descifrarlo, desplazamos cada letra tres posiciones hacia atrás.

F → C, L → I, I → F, U → R, D → A, G → D, R → O.

Resultado: CIFRADO
```

3. Un atacante intercepta un mensaje cifrado con César, pero no sabe el desplazamiento. ¿Cuántas claves tendría que probar como máximo? ¿Qué nos dice esto sobre la seguridad del algoritmo?

El atacante tendría que probar como máximo 25 desplazamientos distintos, sin contar el desplazamiento 0, que no cambia el mensaje.

Esto demuestra que el cifrado César es poco seguro, ya que tiene muy pocas claves posibles y se puede romper fácilmente probándolas todas.

4. Según el **principio de Kerckhoffs**, ¿qué debe ser secreto: el algoritmo o la clave? Explica por qué.

Según el principio de Kerckhoffs, lo que debe mantenerse en secreto es la clave, no el algoritmo.

El algoritmo puede ser conocido públicamente y seguir siendo seguro si la clave se mantiene en secreto. De esta forma, la seguridad no depende de ocultar cómo funciona el sistema.

Ejercicio 2 · Cifrado simétrico: claves y algoritmos

Dificultad: Media · Tiempo orientativo: 30 min

1. En el cifrado simétrico, cada pareja de personas que quiere comunicarse en secreto necesita su propia clave. Calcula cuántas claves hacen falta en una empresa de:

La fórmula para calcular el número de claves es:

$$\frac{n(n-1)}{2}$$

a) Para 4 empleados:

$$\frac{4(4-1)}{2}=\frac{12}{2}=6$$

Se necesitan 6 claves.

b) Para 10 empleados:

$$\frac{10(10-1)}{2}=\frac{90}{2}=45$$

Se necesitan 45 claves.

c) Para 100 empleados:

$$\frac{100(100-1)}{2}=\frac{9900}{2}=4950$$

Se necesitan 4.950 claves.

Pista: la fórmula es n·(n−1)/2. ¿Qué problema de la criptografía simétrica pone en evidencia este cálculo?

Problema que pone en evidencia: La gestión y distribución de claves se vuelve complicada cuando aumenta el número de usuarios. Cada pareja necesita una clave compartida diferente, por lo que la cantidad de claves crece rápidamente.

2. Completa la tabla de algoritmos simétricos:

Algoritmo	Bloque o flujo	Tamaño de clave	¿Se recomienda hoy?
DES	Bloques	56 bits efectivos	No, es inseguro.
3DES	Bloques	112 o 168 bits nominales, según la variante	No, está obsoleto.
AES	Bloques	128, 192 o 256 bits	Sí, es uno de los estándares actuales.
Blowfish	Bloques	De 32 a 448 bits	No se recomienda para sistemas nuevos; utiliza bloques de solo 64 bits.
ChaCha20	Flujo	256 bits	Sí, es un algoritmo moderno y seguro cuando se utiliza correctamente.

3. Explica la diferencia entre cifrar **por bloques** y cifrar **en flujo**.

El cifrado por bloques divide los datos en bloques de tamaño fijo y cifra cada bloque siguiendo el modo de operación correspondiente. Un ejemplo es AES, que utiliza bloques de 128 bits.

El cifrado en flujo cifra los datos progresivamente, generando una secuencia de cifrado que se combina con el mensaje original. Un ejemplo es ChaCha20.

4. ¿Qué problema tiene el modo **ECB**? ¿Cómo lo resuelve el modo **CBC**?

El modo ECB cifra cada bloque de forma independiente. Si dos bloques de texto en claro son iguales, producen bloques cifrados iguales. Esto puede revelar patrones del mensaje original.

El modo CBC soluciona este problema combinando cada bloque de texto en claro con el bloque cifrado anterior antes de cifrarlo. El primer bloque utiliza un vector de inicialización (IV). Así, los bloques iguales del mensaje original normalmente producen bloques cifrados diferentes.

Ejercicio 3 · Laboratorio: cifrado simétrico con OpenSSL

Dificultad: Avanzada · Tiempo orientativo: 40 min

Trabaja en una máquina Linux (o WSL). Ejecuta los comandos y responde a las preguntas con capturas.

Parte A · Cifrar y descifrar un fichero con AES-256

```
echo "Informe confidencial 2 ASIR" > secreto.txt
openssl enc -aes-256-cbc -pbkdf2 -salt -in secreto.txt -out secreto1.enc
openssl enc -aes-256-cbc -pbkdf2 -salt -in secreto.txt -out secreto2.enc
od -c secreto1.enc | head -2
openssl enc -d -aes-256-cbc -pbkdf2 -in secreto1.enc -out recuperado.txt
cat recuperado.txt
```

1. Cifra dos veces con **la misma contraseña**. Compara `secreto1.enc` y `secreto2.enc` (`cmp secreto1.enc secreto2.enc`). ¿Son iguales? ¿Por qué?

No, los ficheros son diferentes porque OpenSSL genera una sal aleatoria en cada cifrado. Esta sal hace que, aunque se utilicen el mismo texto y la misma contraseña, el resultado cifrado sea diferente.

2. ¿Qué texto aparece al principio del fichero cifrado con `od -c`? ¿Para qué sirve?

Al principio aparece normalmente la cadena `Salted__`. Sirve para identificar el formato de OpenSSL e indicar que a continuación se almacena la sal utilizada para derivar la clave y el vector de inicialización. La sal no es secreta.

3. Intenta descifrar con una contraseña incorrecta. ¿Qué ocurre?

El descifrado no recupera el texto original. Puede producir un error de relleno o generar datos incorrectos, dependiendo del resultado. AES-CBC por sí solo no garantiza que una contraseña incorrecta se detecte siempre de forma fiable.

4. Compara el tamaño del original y del cifrado (`ls -l`). ¿Por qué crece?

El fichero cifrado ocupa más espacio debido a la información adicional, como la cabecera y la sal, y al relleno necesario para completar los bloques de AES. El tamaño exacto depende de la longitud del texto original y del formato empleado por OpenSSL.

Parte B · ECB frente a CBC

```
printf 'ASIR-ASIR-ASIR!!%.0s' 1 2 3 4 > bloques.txt
openssl enc -aes-128-ecb -nosalt -K 00112233445566778899aabbccddeeff \
        -in bloques.txt -out ecb.bin
openssl enc -aes-128-cbc -nosalt -K 00112233445566778899aabbccddeeff \
        -iv 0102030405060708090a0b0c0d0e0f10 -in bloques.txt -out cbc.bin
od -A d -t x1 -v ecb.bin
od -A d -t x1 -v cbc.bin
```

5. ¿Cuántos bytes tiene `bloques.txt`? ¿Cuántos bloques de AES son?

El fichero tiene 64 bytes. Como AES utiliza bloques de 16 bytes, contiene cuatro bloques de datos originales.

6. Observa la salida de `ecb.bin`. ¿Qué notas en las primeras cuatro líneas? ¿Y en `cbc.bin`?

En `ecb.bin`, los cuatro primeros bloques cifrados son iguales porque los cuatro bloques de texto en claro son idénticos y ECB cifra cada bloque de manera independiente.

En `cbc.bin`, los bloques cifrados son diferentes porque CBC combina cada bloque con el bloque cifrado anterior antes de cifrarlo. El primer bloque utiliza el vector de inicialización

7. ¿Para qué sirve el parámetro `-iv` en CBC?

El parámetro `-iv` establece el vector de inicialización que se utiliza para cifrar el primer bloque en modo CBC. Ayuda a evitar que mensajes iguales cifrados con la misma clave produzcan necesariamente el

mismo primer bloque. El IV no tiene que ser secreto, pero debe elegirse correctamente y no reutilizarse de forma inadecuada.

8. ¿Por qué los ficheros cifrados tienen 80 bytes y no 64?

Los ficheros cifrados tienen 80 bytes porque AES trabaja con bloques de 16 bytes y OpenSSL utiliza relleno para completar los bloques. Como el texto original ya ocupa exactamente 64 bytes, se añade un bloque completo de relleno de 16 bytes.